

Tarea #4: Pipeline Autónomo de Detección Auto-etiquetado con Qwen VL, Transfer Learning y Validación Reporte Técnico

Armando Rosas Alexandro Ruiz José Camarillo Pedro Avila

Agosto 2026

Repositorio: <https://github.com/armandorosashdz/husky-detection>

Índice

1. Objetivo de la Tarea	2
2. Metodología	2
2.1. Fase 1: Recolección y auto-etiquetado con Qwen3.5	2
2.2. Fase 2: Transfer Learning de YOLOv8s	4
2.3. Fase 3: Despliegue del detector	4
2.4. Fase 4: Validación híbrida en cascada	5
2.5. Fase 5: Métricas	5
3. Resultados	5
3.1. Tabla comparativa (conjunto de prueba, 30 imágenes / 60 cajas)	5
3.2. Curvas de entrenamiento (Fase 2)	6
3.3. Matriz de confusión (conjunto de prueba, 30 imágenes)	7
3.4. Curvas Precision-Recall @0.5 IoU (conjunto de prueba)	8
3.5. Análisis de resultados visuales	9
4. Cuestionario de Análisis Crítico	11
5. Conclusiones	13

1. Objetivo de la Tarea

El objetivo de este proyecto fue diseñar e implementar un pipeline moderno de visión computacional capaz de resolver el problema de escasez de datos etiquetados mediante el uso de un Modelo de Lenguaje y Visión (VLM) de arquitectura abierta, **Qwen3.5**. El pipeline cubre cuatro fases: (1) recolección y auto-etiquetado de 100 imágenes de perros husky mediante Qwen3.5, (2) ajuste fino (*transfer learning*) de un detector YOLOv8s sobre las anotaciones generadas automáticamente, (3) un script de despliegue optimizado para inferencia, y (4) una etapa de validación híbrida en cascada, donde cada detección de YOLOv8s es confirmada o descartada por el mismo VLM en dos tamaños distintos (0.8B y 2B), permitiendo comparar el costo computacional contra la mitigación de falsos positivos que aporta cada uno.

2. Metodología

2.1. Fase 1: Recolección y auto-etiquetado con Qwen3.5

Se recolectaron 100 imágenes de internet con múltiples instancias de perros husky por imagen. Antes de procesarlas, se normalizó su nomenclatura al esquema `husky_000.jpg ... husky_099.jpg` y se redujo el tamaño de cualquier imagen que excediera 1280px en su lado más largo (20 de las 100 imágenes lo excedían). Esta normalización de tamaño resultó necesaria en la práctica: varias imágenes originales alcanzaban resoluciones de hasta 5616×3744px, y el codificador visual de Qwen convierte los píxeles de la imagen en “tokens visuales” cuyo costo de atención escala de forma cuadrática — una imagen sobredimensionada podía hacer que una sola llamada de `generate()` intentara reservar más de 10 GB de memoria de golpe, provocando un *Out of Memory* en GPU.

Selección del modelo. Se utilizó **Qwen3.5**, en su versión **2B** (Qwen/Qwen3.5-2B) para el etiquetado. La corrida completa de la versión final del pipeline (rama `main`: Fases 1 a 4, incluyendo el re-etiquetado con verificación de raza y el reentrenamiento de YOLOv8s) se hizo localmente en la siguiente laptop:

Cuadro 1: Especificaciones de hardware y software utilizadas

Componente	Especificación
Sistema operativo	Windows 10
Procesador	Intel Core i7-12650HX (12. ^a generación)
RAM	16 GB
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
VRAM	6.0 GB
CUDA	12.1
PyTorch	2.5.1+cu121
Python	3.10.20 (conda-forge)
Ultralytics	8.4.95

Prompt de detección. Se diseñó un prompt específico y estricto que pide explícitamente *todas* las instancias de husky visibles, cajas ajustadas sin adivinar partes ocluidas, y una salida JSON sin texto adicional:

```
"Locate every single husky dog visible in this image, even if there are several of them. Do not stop after the first one -- include ALL husky dogs you can see as separate entries in the array. Output ONLY a JSON array, no explanation, no markdown fences, no bold/asterisks, in this exact format: [{"bbox_2d": [x1, y1, x2, y2], "label": "husky dog"}, ...] where coordinates are normalized to a 0-1000 scale relative to the image width and height [...]"
```

Esta versión reemplazó a un prompt inicial más corto (conservado comentado en `config.py` como evidencia para el análisis de sensibilidad de la Sección 4), que en ocasiones se detenía tras detectar el primer perro o variaba el formato de salida.

Formato de salida de Qwen. A pesar de que el prompt original pedía el orden `[ymin, xmin, ymax, xmax]` (siguiendo el ejemplo del enunciado), se verificó experimentalmente — dibujando ambas interpretaciones sobre imágenes de prueba — que Qwen3.5 ignora esa instrucción y responde siempre en su esquema nativo de *grounding*: `{"bbox_2d": [x1, y1, x2, y2], "label": ...}`, en escala 0–1000. El prompt final se ajustó para pedir directamente ese formato nativo en lugar de forzar el orden del enunciado, y la conversión matemática a formato YOLO (`class_id x_center y_center width height` normalizado 0–1) se implementó de acuerdo a este orden confirmado.

Modo de razonamiento (*thinking mode*). Qwen3.5-4B/9B activan por defecto un modo de razonamiento que antepone un bloque `<think>...</think>` a la respuesta — un problema descubierto en una etapa anterior del proyecto, cuando el etiquetado de Fase 1 se probó con el modelo de 4B: el límite de `max_new_tokens` se agotaba dentro del bloque de razonamiento antes de llegar a las cajas, dando la falsa impresión de que el modelo era incapaz de detectar los perros. Se corrigió pasando `enable_thinking=False` al `apply_chat_template`, además de un mecanismo de defensa adicional que elimina cualquier bloque `<think>...</think>` antes de buscar el JSON de salida. El etiquetador usado en la versión final (**Qwen3.5-2B**) no activa este modo por defecto, por lo que el *bug* original no se manifestó en esta corrida, pero `enable_thinking=False` se dejó activo de forma incondicional en el código como medida defensiva, sin costo para tamaños que no lo necesitan.

Cada imagen procesada generó un archivo `.txt` en formato YOLO y una visualización con las cajas dibujadas para revisión manual (`data/labels_check/`). Las 100 imágenes fueron etiquetadas exitosamente.

Verificación de raza (VERIFY_BREED), agregada tras revisar los resultados de validación.

Un análisis exploratorio hecho por el equipo sobre un holdout de imágenes separado (descontinuado en la versión final del pipeline, que evalúa contra el conjunto de prueba en su lugar) encontró casos donde el *ground truth* pseudo-etiquetado en esta fase incluía perros de otras razas visualmente similares (notablemente Alaskan Malamute) junto a huskies genuinos — el validador de la cascada (Fase 4) a veces rechazaba correctamente esos casos, exponiendo el sesgo, y otras veces lo heredaba igual. Para mitigar esto en la versión final del pipeline (rama `main`), se añadió un segundo paso de verificación: cada caja que Qwen detecta se recorta y se le hace una consulta binaria adicional (`PROMPT_VERIFY_BREED`, reutilizando el mismo modelo ya cargado como `QWEN_LABELER`) preguntando específicamente si el recorte muestra un Husky Siberiano y no otra raza ni un lobo; las cajas que no se confirman se descartan antes de escribirse. `PROMPT_LABELING` también se endureció para excluir explícitamente otras razas, como defensa adicional (no reemplaza la verificación por recorte, ya que Qwen no siempre respeta esa instrucción al 100 %). El mismo prompt estricto se reutilizó en `PROMPT_VALIDATION` (Fase 4), reemplazando la pregunta genérica original (“Is this a husky dog...”) por una que también excluye razas similares y lobos.

2.2. Fase 2: Transfer Learning de YOLOv8s

Las 100 imágenes etiquetadas se dividieron en un 70 % de entrenamiento (70 imágenes) y 30 % de prueba (30 imágenes), con semilla fija (`SEED=42`) para reproducibilidad, generando el archivo `dataset.yaml` requerido por Ultralytics.

Augmentación de datos. Dado el bajo volumen de datos, se configuró augmentación agresiva: `mosaic=1.0`, `mixup=0.15`, `hsv_h=0.015`, `hsv_s=0.7`, `hsv_v=0.4`, `fliplr=0.5`, `degrees=10.0`, `scale=0.5`.

Corrección de inestabilidad en el entrenamiento. El primer intento real de entrenamiento (100 imágenes, 100 épocas, GPU Kaggle) mostró una época 1 prometedora (`mAP50= 0,71`), pero la pérdida de clasificación (`cls_loss`) explotó de 3.5 a 36 hacia la época 8 y el modelo nunca se recuperó. La causa fue que `optimizer="auto"` (el valor por defecto de Ultralytics) eligió AdamW con `lr=0.002`, demasiado agresivo para solo 70 imágenes (9 *batches* por época) — y al usar `optimizer="auto"`, Ultralytics *ignora* cualquier `lr0` personalizado que se le pase. Se corrigió fijando el optimizador explícitamente (`OPTIMIZER="AdamW"`) y congelando las primeras 10 capas del *backbone* (`FREEZE=10`) para un *transfer learning* más estable con pocos datos. LRO se fijó inicialmente en 0.001 (la mitad del valor que “auto” había elegido); en una corrida posterior con un conjunto de entrenamiento distinto el mismo colapso de `cls_loss` volvió a aparecer, así que se redujo de nuevo a la mitad (`LRO=0.0005`) y se subió `PATIENCE` de 20 a 60 épocas, dando más margen para recuperarse antes de que el *early stopping* se quede con un *checkpoint* temprano y poco entrenado como “mejor”.

Decisión de diseño: clase única, sin preservar clases de COCO. El enunciado sugiere mantener las demás clases preentrenadas activas. Sin embargo, el ajuste fino estándar de Ultralytics con `nc=1` reinicializa la cabeza de detección, perdiendo las 80 clases de COCO (*catastrophic forgetting*) incluso congelando el *backbone*, ya que los pesos no son específicos por clase. Existe una técnica para preservar genuinamente las clases originales (congelar las primeras 22 capas, entrenar una segunda cabeza de detección solo para la clase nueva y fusionar ambas salidas con una capa `ConcatHead` personalizada), pero requiere modificar la librería `ultralytics` y se recomienda para conjuntos de 1000+ imágenes de la clase nueva — muy por encima de las 70 disponibles. Dado el alcance académico, sin GPU local y con tiempo limitado, se optó por el enfoque estándar (`nc=1`, una sola clase, se pierden las clases de COCO), documentado aquí como una limitación deliberada y no un descuido.

2.3. Fase 3: Despliegue del detector

Se implementó `hybrid_inference.py`, un script de inferencia que carga los pesos entrenados de YOLOv8s (`YOLODetector`) y procesa un flujo de imágenes midiendo latencia y FPS reales por imagen, simulando un escenario de producción. El umbral de confianza se dejó deliberadamente bajo para maximizar el número de detecciones que llegan a la cascada de validación de la Fase 4, tal como recomienda el enunciado — inicialmente en `CONF_THRESHOLD=0.15`, pero se observó que el *recall* tenía un techo duro de ~ 0.70 (cerca de un 30 % de los huskies del *ground truth* nunca alcanzaba esa confianza), a pesar de que el modelo entrenado era capaz de `mAP@0.5= 0,937` según Ultralytics. Se bajó a `CONF_THRESHOLD=0.01` en la versión final, dejando pasar muchas más detecciones de baja confianza (y por tanto más falsos positivos) a la cascada, bajo la premisa de que es la cascada la que debe filtrarlos, no el umbral del detector.

2.4. Fase 4: Validación híbrida en cascada

Cada detección de YOLOv8s se recorta (con un margen de 10px, `CROP_PADDING`) y se envía a Qwen VL con un prompt binario de confirmación. La versión inicial del prompt (`Is this a husky dog inside this image crop? Answer only Yes or No.`) no excluía explícitamente otras razas ni lobos, dejando pasar más falsos positivos de raza en la cascada de los que debería; en la versión final se endureció al mismo texto usado en `PROMPT_VERIFY_BREED` (Fase 1):

```
"Look only at this cropped image. Is the dog shown specifically a Siberian Husky
(not any other breed, and not a wolf)? Answer with exactly one word: Yes or No."
```

Solo las detecciones cuya respuesta comienza con “Yes” se conservan; el resto se descarta como falso positivo. Tras la cascada se aplica una segunda pasada de tipo *NMS* (`deduplicate_boxes`) para eliminar cajas redundantes que el validador VLM aprobó por separado sobre el mismo perro — el VLM responde “sí es un husky” por recorte, sin saber que otra caja ya aprobada cubre al mismo animal.

Siguiendo el requisito de comparación de la tarea, el pipeline se ejecutó de manera idéntica con dos tamaños de validador: **Qwen3.5-0.8B** y **Qwen3.5-2B**, sobre el mismo *hardware* en cada corrida (para que la comparación de latencia fuera válida) — la laptop descrita en la Tabla 1.

2.5. Fase 5: Métricas

Las métricas (mAP@0.5, precisión, *recall*, falsos positivos/negativos, latencia y FPS) se calculan con `metrics.py`: coincidencia de detecciones contra *ground truth* por IoU (umbral 0.5), y el cómputo final de AP se delega en `ultralytics.utils.metrics.compute_ap` (interpolación de 101 puntos estilo COCO) en lugar de una implementación manual estilo VOC, tras confirmar que ambas dan resultados consistentes (diferencia de ~ 0.003 en mAP@0.5 sobre las mismas detecciones).

3. Resultados

3.1. Tabla comparativa (conjunto de prueba, 30 imágenes / 60 cajas)

El conjunto de prueba (`data/test/`) corresponde al 30 % de las 100 imágenes originales que la Fase 2 nunca usó para entrenar (las mismas 30 imágenes de siempre — el *split* usa una semilla fija). Los números de esta sección corresponden a la **versión final del pipeline** (rama `main`: verificación de raza en Fase 1, `CONF_THRESHOLD=0.01`, prompt de validación endurecido, modelo re-entrenado), corridos localmente por el equipo; el número de cajas de *ground truth* bajó de 79 a **60** frente a la versión anterior del reporte, porque la verificación de raza filtró del re-etiquetado varios perros que no eran huskies puros. Las tres configuraciones se evaluaron sobre el mismo *hardware*, por lo que la latencia y los FPS son directamente comparables entre sí.

Configuración	mAP@0.5	Precisión	Recall	FP	FN	Latencia (ms)	FPS
YOLOv8s Ajustado (solo)	0.9350	0.4754	0.9667	64	2	23.41	42.72
YOLOv8s + Qwen 0.8B	0.9376	0.6706	0.9500	28	3	2198.84	0.45
YOLOv8s + Qwen 2B	0.9549	0.6667	0.9667	29	2	2328.17	0.43

Cuadro 2: Comparativa de las 3 configuraciones del pipeline sobre el conjunto de prueba (60 cajas de *ground truth* en 30 imágenes), rama `main`. FP/FN son conteos absolutos, no porcentajes.

Con `CONF_THRESHOLD` bajado a 0.01, YOLOv8s solo genera muchas más detecciones de baja confianza que antes, y su precisión cae en consecuencia (0.4754, 64 falsos positivos) — esto es esperado y deliberado: el umbral bajo existe justamente para que sea la cascada la que filtre, no el detector. Ambos validadores cumplen ese rol de forma contundente, reduciendo los falsos positivos de 64 a 28 y 29 respectivamente (más de la mitad), mientras el *recall* se mantiene prácticamente igual al del detector solo. El validador de 2B logra el $\text{mAP}@0.5$ más alto de los tres (0.9549) y el mismo número de falsos negativos que la línea base (2, el mínimo posible entre las tres configuraciones), superando ligeramente al de 0.8B en precisión agregada; el de 0.8B tiene un falso positivo menos (28 vs. 29). El costo de activar cualquiera de los dos validadores sigue siendo una caída de FPS de más de $90\times$ frente al detector solo (ver Sección 4, pregunta 1).

3.2. Curvas de entrenamiento (Fase 2)

El entrenamiento de la versión final corrió las **100 épocas completas** (*patience*=60, sin activarse el *early stopping* esta vez) sobre el conjunto de entrenamiento re-etiquetado con verificación de raza, alcanzando $\text{mAP}@0.5 \approx 0,958$ y $\text{mAP}@0.5:0.95 \approx 0,51$ en el conjunto de validación interno de Ultralytics (`data/test/`, usado también como `val` durante el entrenamiento).

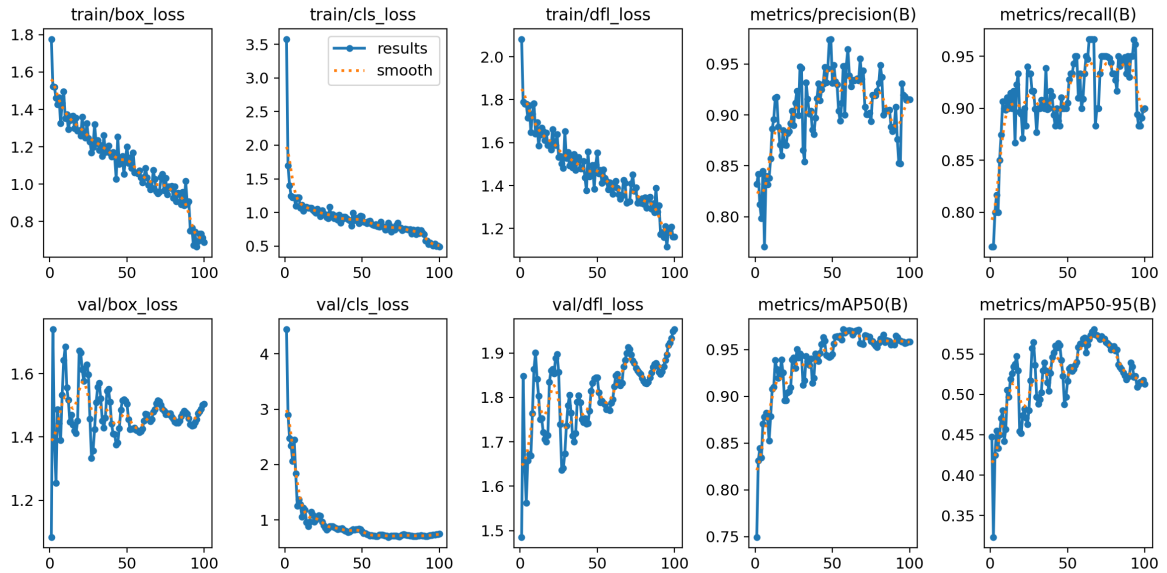


Figura 1: Curvas de pérdida (*box_loss*, *cls_loss*, *dfl_loss*) y de precisión/*recall*/mAP@0.5 durante las 95 épocas de entrenamiento.

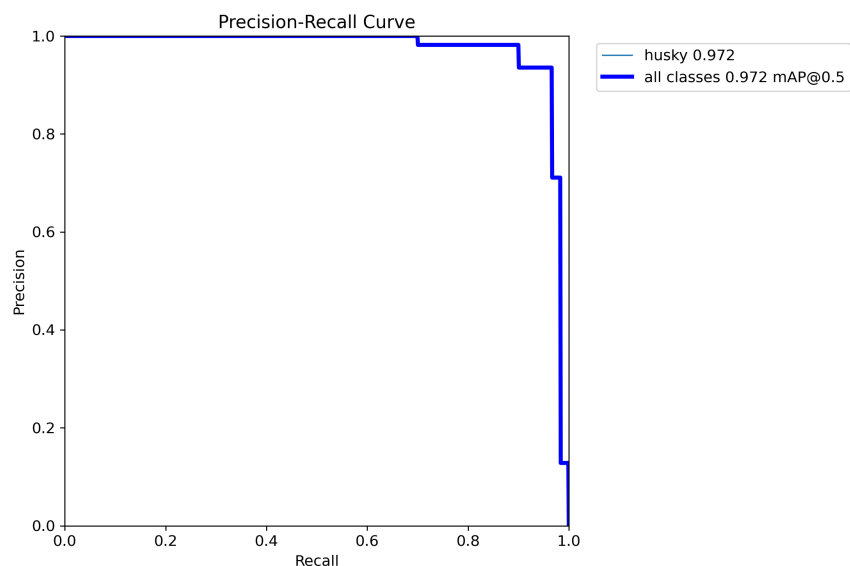


Figura 2: Curva Precision-Recall generada por Ultralytics sobre el conjunto de validación interno al finalizar el entrenamiento.

3.3. Matriz de confusión (conjunto de prueba, 30 imágenes)

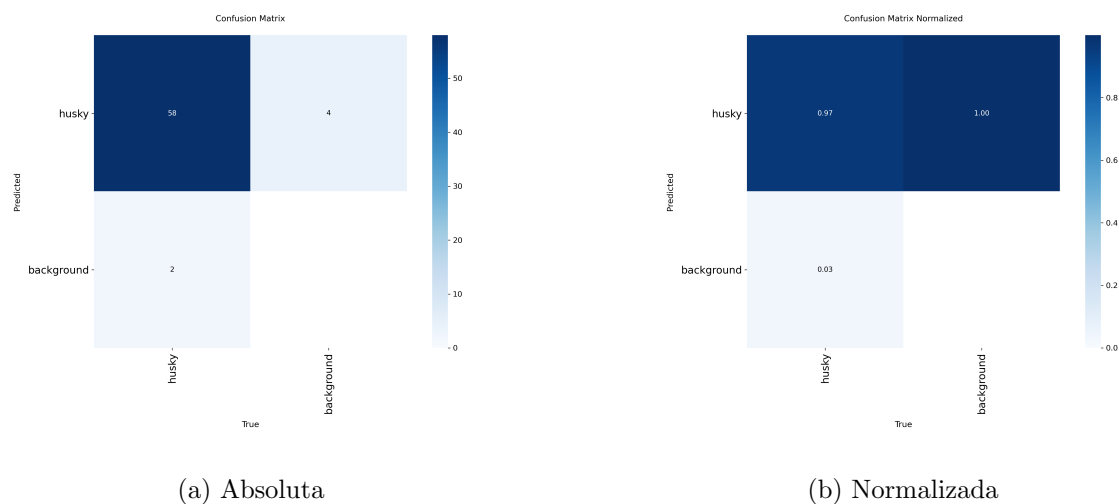


Figura 3: Matriz de confusión de YOLOv8s sobre el conjunto de prueba (30 imágenes), generada por Ultralytics al finalizar el entrenamiento.

3.4. Curvas Precision-Recall @0.5 IoU (conjunto de prueba)

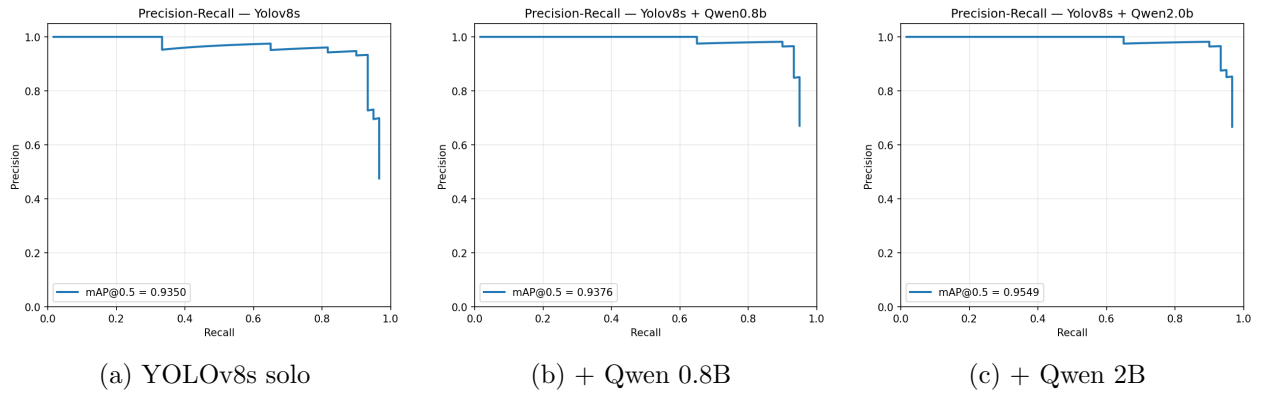


Figura 4: Curvas Precision-Recall individuales de las 3 configuraciones sobre el conjunto de prueba.

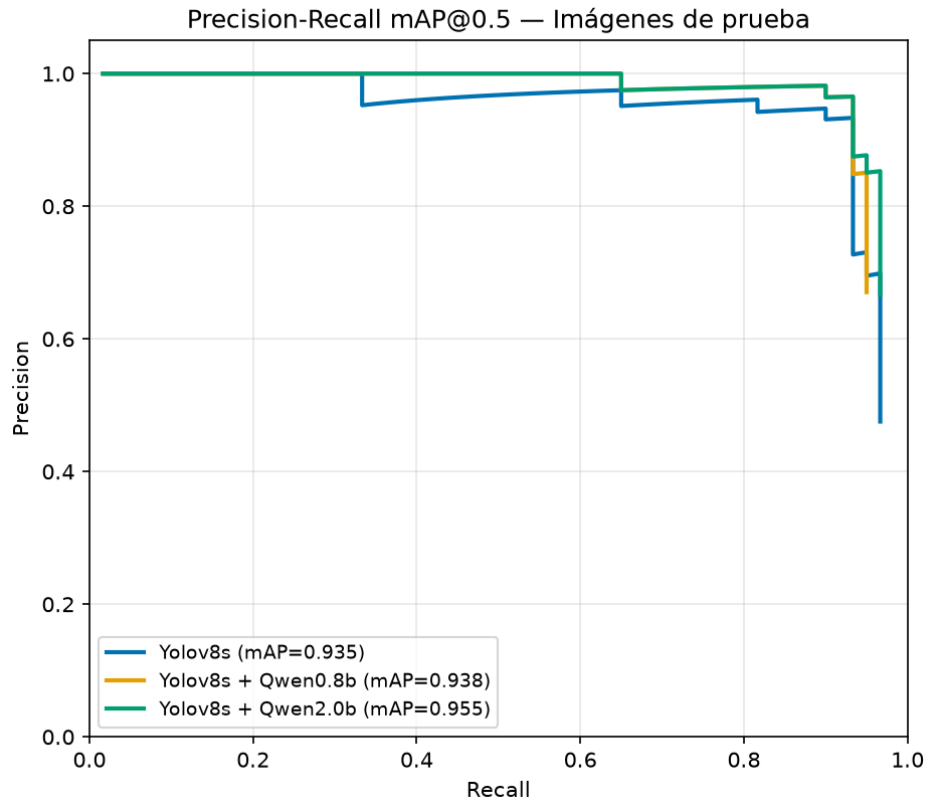


Figura 5: Las 3 curvas Precision-Recall superpuestas para comparación directa (conjunto de prueba).

3.5. Análisis de resultados visuales

La Figura 6 muestra la misma imagen del conjunto de prueba procesada por las 3 configuraciones: las cajas verdes son verdaderos positivos (coinciden con *ground truth*) y las naranjas son falsos positivos. Se observa cómo la cascada de validación reduce las cajas naranjas frente al detector solo.

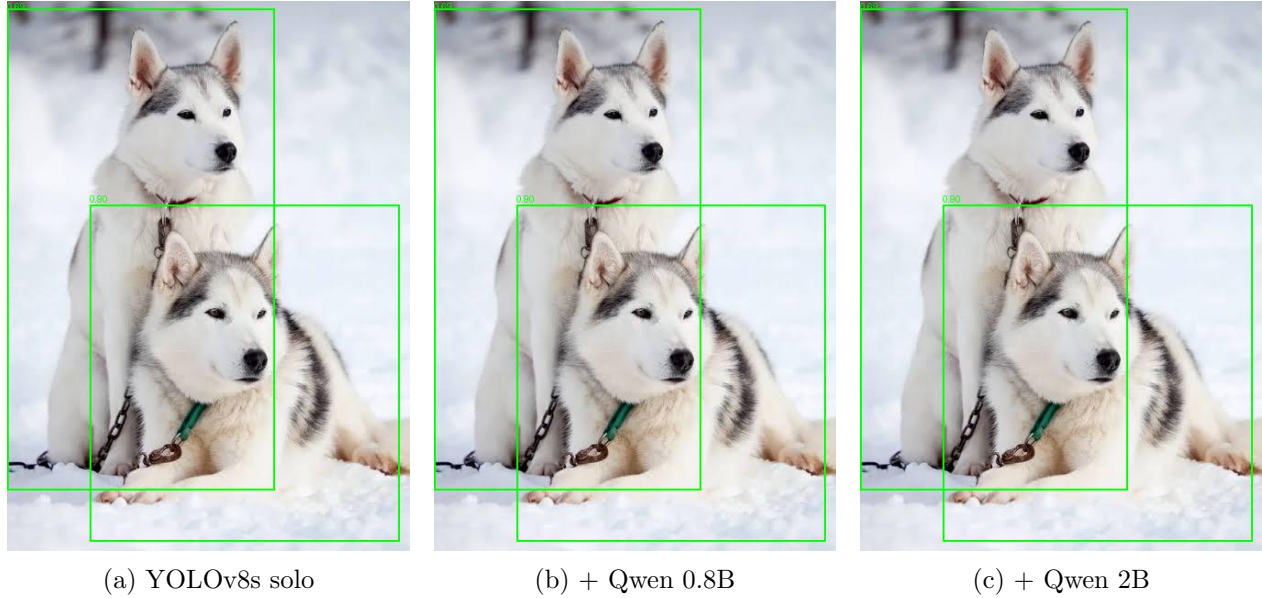


Figura 6: `husky_000.jpg` (conjunto de prueba) procesada por las 3 configuraciones. Verde = verdadero positivo, naranja = falso positivo.

La Figura 7 complementa el ejemplo anterior con 3 casos elegidos a mano para cubrir cada categoría pedida:

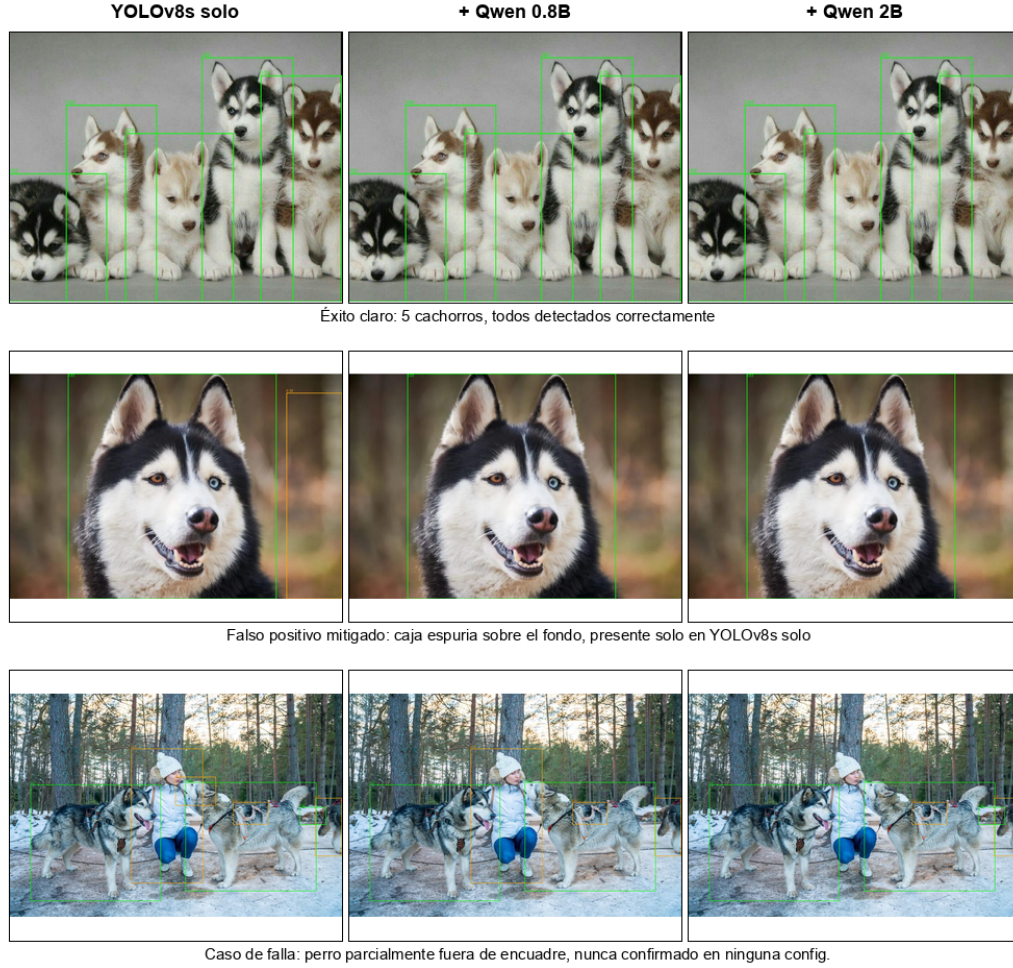


Figura 7: Tres casos curados sobre el conjunto de prueba. Verde = verdadero positivo, naranja = falso positivo.

Éxito claro (*husky_025.jpg*): 5 cachorros en fila, todos detectados con cajas ajustadas en las 3 configuraciones — ningún falso positivo ni falso negativo, y la cascada no introduce ni quita nada sobre una detección ya correcta.

Falso positivo mitigado por la cascada (*husky_003.jpg*): YOLOv8s solo detecta correctamente al husky (verde, conf. 0.93) pero también genera una caja espuria sobre el fondo desenfocado (naranja, conf. 0.18, sin ningún perro dentro). Ambas versiones de la cascada — 0.8B y 2B — rechazan esa caja al preguntarle a Qwen si el recorte contiene un husky, dejando únicamente la detección correcta.

Caso de falla (*husky_029.jpg*): imagen con varios perros, una persona en primer plano y un perro adicional cortado por el borde derecho del encuadre. YOLOv8s solo genera, además de las detecciones correctas, una caja falsa sobre la persona (naranja, conf. 0.13) — el validador de 2B la rechaza correctamente, pero el de 0.8B *no*, mostrando que ninguno de los dos validadores es perfecto. Más relevante como caso de falla: el perro parcialmente visible en el borde derecho nunca recibe una confirmación sólida en ninguna de las 3 configuraciones (permanece en naranja, confianza ≤ 0.05) — una limitación consistente del pipeline ante objetos truncados por el encuadre, no resuelta por ninguna versión de la cascada.

4. Cuestionario de Análisis Crítico

1. Penalización de recursos computacionales

La activación de Qwen VL como validador en cascada degrada el FPS de forma drástica: de **42.72 FPS** (YOLOv8s solo) a **0.45 FPS** con el validador de 0.8B y **0.43 FPS** con el de 2B en el conjunto de prueba — una caída superior a $90\times$. Resulta notable que la diferencia de tamaño entre 0.8B y 2B ($2.5\times$ más parámetros) prácticamente no se refleja en el FPS (2198.84 ms vs. 2328.17 ms de latencia promedio, una diferencia de menos del 6 %). Esto indica que, para recortes pequeños de imagen y prompts binarios cortos (`max_new_tokens` reducido para la validación), el cuello de botella no es el cómputo de los parámetros del modelo en sí, sino costos fijos por llamada (carga de la imagen, *overhead* de generación, *tokenization*) que dominan sobre la diferencia de tamaño del modelo.

Con estos números, el pipeline **no es viable** para un sistema de tiempo real crítico como un vehículo autónomo, donde se requieren decenas de FPS con latencias de pocos milisegundos y consecuencias de seguridad ante un retraso. A menos de 1 FPS, cada detección tarda más de 2 segundos en confirmarse, lo cual solo es aceptable en **aplicaciones asíncronas** donde la latencia no es crítica: por ejemplo, etiquetado *offline* de un archivo de imágenes, un sistema de moderación de contenido que procesa una cola en segundo plano, o una alerta de vigilancia que no necesita confirmarse en tiempo real. Para un caso de uso en tiempo real, sería preferible usar solo YOLOv8s (~ 43 FPS) y aceptar la tasa de falsos positivos considerablemente mayor, o explorar la alternativa de un clasificador ligero descrita en la pregunta 4.

2. Propagación del error

Un error sistemático de desalineación en las cajas generadas por el VLM durante la Fase 1 se propaga directamente a la función de pérdida de regresión de YOLOv8s (*Bounding Box Loss*, compuesta por *box_loss*/CIoU y *dfl_loss* en la implementación de Ultralytics). Si las cajas de *ground truth* están sistemáticamente desplazadas o mal dimensionadas (por ejemplo, consistentemente más anchas de lo real, o recortando extremidades ocluidas), la red *aprende ese sesgo como si fuera la verdad*: el optimizador ajusta los pesos para minimizar la distancia a cajas que ya están mal ubicadas, por lo que el modelo entrenado reproduce el mismo error sistemático en sus propias predicciones, en lugar de aprender la localización real del objeto. Un error *aleatorio* (ruido sin sesgo consistente) tendería a promediarse y afectar menos al modelo; un error *sistemático* (sesgo direccional) degrada la precisión de localización de forma persistente, ya que no hay señal contradictoria en los datos que lo corrija.

Un riesgo análogo puede darse no solo en la localización sino también en la *clase*: si el pseudo-*ground truth* de la Fase 1 incluyera perros de otras razas visualmente similares (notablemente Alaskan Malamute) etiquetados como “husky”, ese sesgo se heredaría directamente en la evaluación posterior, sin que el pipeline tenga forma de detectarlo por sí solo. Este riesgo motivó incorporar el paso de verificación de raza (`VERIFY_BREED`) descrito en la Sección 3.1: cada caja detectada se somete a una consulta adicional que confirma específicamente la raza antes de aceptarla, en vez de confiar únicamente en que el prompt de detección inicial la excluya.

¿Es necesario que un humano revise las anotaciones automáticas? Sí. El propio proyecto incorporó un paso de verificación visual obligatorio (`data/labels_check/`) precisamente porque el auto-etiquetado no es infalible: en la práctica se hicieron ajustes de prompt para reducir errores comunes (cajas flojas, perros no detectados, formato de salida inconsistente), y el hallazgo de sesgo de raza mencionado arriba muestra que ese tipo de error puede sobrevivir a la revisión automática

si no se revisa específicamente para ello. Un humano no necesita re-etiquetar todo el conjunto desde cero, pero sí debe hacer una revisión de muestreo dirigida a detectar sesgos sistemáticos (no solo cajas mal ubicadas) antes de que se propaguen al entrenamiento.

3. Sensibilidad del *prompt engineering*

El proyecto documentó explícitamente dos versiones del prompt de auto-etiquetado (conservadas ambas en `config.py` como evidencia). La primera versión, más corta y genérica (“Detect all husky dogs in this image. Return a JSON list...”), en ocasiones se detenía tras detectar solo el primer perro de la imagen o variaba el formato de salida (agregando texto explicativo o *fences* de markdown alrededor del JSON), lo cual rompía el parseo automático. La versión final, más larga y prescriptiva — que insiste explícitamente en no detenerse tras la primera instancia, pide cajas ajustadas sin adivinar partes ocluidas, y prohíbe explícitamente cualquier formato adicional — redujo notablemente esa varianza. Esto confirma que Qwen3.5 es **sensible a la formulación exacta del prompt**, especialmente en instrucciones de completitud (“todas las instancias”) y de formato de salida estricto: dejar espacio de interpretación se traduce directamente en inconsistencia en las coordenadas y en fallas de parseo, no solo en la calidad semántica de la detección.

Sobre el prompt de validación binaria (Fase 4): funcionó de forma consistente y sin ambigüedad para *ambos* tamaños de modelo en cuanto a formato (respuesta “Yes”/“No” bien formada en prácticamente todos los casos, sin necesidad de *parsing* adicional más allá de verificar el prefijo). En cuanto a *calibración*, la Tabla 2 muestra que el validador de 2B alcanza un mAP@0.5 ligeramente superior (0.9549 vs. 0.9376) y el mismo número de falsos negativos que el de 0.8B, a costa de un falso positivo adicional (29 vs. 28): una diferencia marginal que sugiere que, con el prompt final empleado, ambos tamaños de modelo aplican un criterio de decisión comparable sobre las mismas instrucciones, sin que uno domine claramente sobre el otro.

4. Optimización arquitectónica

Reemplazar el validador VLM por un clasificador convolucional ligero entrenado desde cero (p. ej. ResNet18 o MobileNet) sobre los recortes de detección tiene un argumento de eficiencia computacional muy fuerte a su favor: los datos de este mismo proyecto lo evidencian — un clasificador especializado y pequeño correría en el orden de milisegundos por recorte (similar a la latencia de YOLOv8s, ~23 ms para una imagen completa), frente a los ~2.2–2.3 segundos por imagen que toma la cascada con Qwen VL en este pipeline. La ganancia de FPS potencial es de varios órdenes de magnitud, precisamente lo que se necesitaría para acercarse a un escenario de tiempo real (pregunta 1).

Ventajas del clasificador ligero.

- Latencia y consumo de memoria órdenes de magnitud menores (millones de parámetros vs. cientos de millones/miles de millones en Qwen).
- Al ser un modelo de clasificación binaria dedicado, puede optimizarse/cuantizarse agresivamente (INT8, TensorRT) sin la complejidad de un modelo generativo de lenguaje.
- No depende de que un LLM interprete correctamente un prompt en lenguaje natural — el espacio de salida es fijo, eliminando el riesgo de que el modelo alucine texto plausible pero incorrecto.

Desventajas del clasificador ligero frente al VLM.

- Requiere un conjunto de entrenamiento etiquetado específico (recortes positivos/negativos de husky vs. no-husky) que en este proyecto *no existe* de forma independiente — habría que generarlo, posiblemente con el mismo VLM, reintroduciendo el problema original que motivó usar un VLM en primer lugar.
- Pierde la capacidad de razonamiento semántico general de un VLM: Qwen puede, en principio, justificar o generalizar mejor a variaciones no vistas en entrenamiento (ángulos, oclusiones, fondos inusuales) gracias a su preentrenamiento masivo, mientras que un CNN pequeño entrenado desde cero con pocos datos es más propenso a sobreajustarse a las particularidades del conjunto de entrenamiento.
- El riesgo de sesgo de raza discutido en la pregunta 2 (motivo por el cual se incorporó `VERIFY_BREED`) sugiere que el conocimiento de dominio amplio de un VLM puede, en principio, detectar matices de clase — distinguir un Husky Siberiano de una raza similar como el Alaskan Malamute — que un clasificador binario simple, entrenado solo para “¿es un perro dentro de esta caja?”, no necesariamente capturaría sin datos específicos de esa distinción; replicar esa capacidad en un clasificador ligero requeriría entrenarlo explícitamente para diferenciar razas similares, no solo “perro vs. no-perro”.
- Menos flexible ante cambios de tarea: cambiar el criterio de validación en el VLM es solo cuestión de reescribir el prompt; en un clasificador dedicado requeriría reentrenar.

5. Conclusiones

El pipeline implementado demuestra que es posible construir un detector de objetos razonablemente preciso ($\text{mAP}@0.5 \approx 0,95$ en el conjunto de prueba, con la cascada de 2B) partiendo de cero etiquetas manuales, usando un VLM abierto tanto para generar las anotaciones de entrenamiento como para validar las detecciones en producción. La cascada de validación con Qwen VL cumple su objetivo de reducir falsos positivos de forma contundente (más de la mitad en el conjunto de prueba final), pero a un costo de FPS que la hace inviable para aplicaciones de tiempo real estricto, y su calibración no es perfectamente uniforme entre tamaños de modelo, como se discute en la pregunta 3 del cuestionario. La principal limitación identificada durante el desarrollo del proyecto — sesgo de raza heredado del auto-etiquetado de la Fase 1, expuesto por un análisis exploratorio del equipo — motivó agregar un paso explícito de verificación de raza al pipeline (rama `main`), cuyo efecto ya se refleja en el conjunto de prueba (el *ground truth* se redujo de 79 a 60 cajas al filtrar perros que no eran huskies puros). Cabe notar que la versión final del pipeline evalúa “validación” reutilizando las mismas imágenes del conjunto de prueba en vez de mantener un holdout completamente separado, por lo que las cifras reportadas conservan el ligero sesgo optimista de haber sido usadas también como `val` interno de Ultralytics durante el entrenamiento. Las demás limitaciones documentadas (pérdida de las clases COCO originales, costo computacional de la cascada) son conocidas y estarían mitigadas parcialmente por líneas de trabajo futuro puntuales: preservar clases COCO con una segunda cabeza de detección, mantener un holdout de validación genuinamente separado del conjunto de prueba, y sustituir el validador VLM por un clasificador ligero especializado para casos de uso en tiempo real.